



CIUDAD DE MÉXICO  
CAPITAL DE LA TRANSFORMACIÓN

## DOCUMENTACIÓN TÉCNICA: SISTEMA DE CONTROL Y GESTIÓN DOCUMENTAL

Por :Rayas Calderón Johann Haziél

**Introducción del Desarrollador** Hola, preparé este documento para explicar a detalle la arquitectura, lógica y flujo del Sistema de Control Documental interno que desarrollé para la Coordinación de Administración de Capital Humano. Como estudiante avanzado de Informática en la FES Cuautitlán y desarrollador full-stack, diseñé este sistema enfocándome en **resolver** un problema real de la administración pública: el rastreo, asignación y validación de **oficios**.

Mi objetivo principal fue crear una herramienta robusta pero intuitiva, asegurando que el flujo de trabajo entre la Coordinación, las Jefaturas de Unidad Departamental (JUDs) y el personal operativo sea transparente y medible.

A continuación, detallo cómo estructuré el proyecto.

### 1. Arquitectura y Stack Tecnológico

Para este proyecto decidí mantener una arquitectura monolítica clásica enfocada en la velocidad de desarrollo y facilidad de despliegue en servidores convencionales:

- **Backend:** PHP (Nativo/Estructurado). Utilicé sesiones (`$_SESSION`) para mantener el estado del usuario y reglas lógicas directas en la cabecera de cada archivo.
- **Base de Datos:** MySQL. Toda la interacción la realizo a través de **PDO (PHP Data Objects)** con sentencias preparadas (`prepare` y `execute`) para garantizar la seguridad contra inyecciones SQL. Centralicé la conexión en una clase `Conexion` en `conexion.php`.
- **Frontend:** HTML5, CSS3 y JavaScript vanilla.
  - Para la responsividad y estructura base, integré **Bootstrap 5.3** vía CDN.
  - El diseño UI/UX lo personalizé sobrescribiendo variables CSS nativas (`:root`) para adaptar estrictamente la paleta de colores institucionales (tonos Guinda `#9f2241` y Oro `#bc955c`).
  - Uso **Bootstrap Icons** y **FontAwesome** para la iconografía.
  - Tipografía: *Plus Jakarta Sans* para darle un toque moderno y legible a las interfaces densas en datos.

### 2. Control de Acceso Basado en Roles (RBAC)

Diseñé un motor de permisos muy estricto al inicio de cada archivo. Dependiendo del rol (`$_SESSION['rol']`), el sistema muta y muestra interfaces o habilita funciones distintas:

1. **Administrador / Coordinación:** Son los superusuarios. Tienen privilegios para:
  - Turnar y capturar nuevos oficios (con vista de formulario priorizada).



CIUDAD DE MÉXICO  
CAPITAL DE LA TRANSFORMACIÓN

- Acceder al CRUD Global para ediciones destructivas.
  - Dar de alta/baja a empleados operativos.
  - Clonar oficios y ver el monitor estratégico.
2. **Dirección General / Secretaria:** Entran en *Modo Observador*. Pueden ver todos los oficios y reportes para tomar decisiones, pero el sistema les bloquea los botones de captura, edición, validación y gestión de personal.
  3. **JUD (Jefatura de Unidad Departamental):** Acceso restringido. Solo ven los oficios turnados a su área (`id_area`). No pueden capturar oficios nuevos, pero sí pueden asignar la tarea a su personal operativo interno y responder/finalizar trámites.

### 3. Descripción de Módulos Principales

#### 3.1. Tablón de Expedientes (`tablon.php`)

Es el corazón operativo del sistema. Lo dividí dinámicamente en dos columnas si el usuario tiene permisos de captura:

- **Columna Izquierda (Captura):** Un formulario extenso para registrar el oficio. Incluye cálculos de fechas de vencimiento con JavaScript (excluyendo fines de semana si la prioridad es "Con término"), foliado extra (CACH, DGAf, CJSL) y carga de archivos PDF.
- **Columna Derecha (Bandeja):** Renderizo una tabla iterando los resultados de la base de datos. Puse especial atención en la UI visual de los estados: utilizo *badges* de colores para prioridades (Urgente, Extraurgente, etc.) y detecto si un documento está rechazado o esperando validación para resaltarlo visualmente.

#### 3.2. Motor de Procesamiento y Validación (`subir_doc.php`)

Aquí recibo el POST del formulario de captura. Implementé dos validaciones críticas en el backend:

1. **Regla de No Duplicidad:** Antes de hacer cualquier inserción, ejecuto un `SELECT` para verificar si el `num_ingreso` (que no sea "S/N") ya existe en la base de datos. Si existe, aborto la operación y retorno un script de alerta para no sobrescribir ni duplicar oficios.
2. **Validación de Archivos Segura:** No me fío de la extensión del archivo. Uso `finfo_open(FILEINFO_MIME_TYPE)` para leer los bytes reales del archivo subido. Si no es un `application/pdf` auténtico, detengo el proceso. Además, renombro el archivo usando un `timestamp` + `bin2hex` (ej. `17104523_a1b2c3d4.pdf`) para evitar colisiones en el servidor.

#### 3.3. Monitoreo Estratégico (`tablon_monitoreo.php`)



CIUDAD DE MÉXICO  
CAPITAL DE LA TRANSFORMACIÓN

Desarrollé este dashboard pensando en la Dirección. Ejecuto consultas SQL con condicionales `SUM(CASE WHEN...)` para obtener métricas masivas en un solo viaje a la base de datos.

- Muestra tarjetas (KPIs) con totales de folios activos, concluidos y en revisión.
- Construí una tabla de rendimiento que calcula matemáticamente el porcentaje de eficiencia de cada JUD (oficios concluidos vs carga total), renderizando barras de progreso dinámicas (Verdes si es >80%, amarillas >50%, rojas <50%).

### 3.4. Productividad por Empleado (`empleados_tareas.php`)

Este módulo lo hice para auditar cargas de trabajo individuales. Hago un cruce entre la tabla `personal` y `documentos`. El sistema agrupa los oficios por estado (Pendiente, Revisión, Concluido) bajo la tarjeta de cada analista.

- Le añadí una función en JavaScript pura (`imprimirListadoPendientes`) que extrae el HTML específico de los pendientes de un analista, inyecta estilos CSS para impresión, y abre un popup para imprimir un reporte físico al instante.

### 3.5. CRUD Global Avanzado (`crud_global.php`)

Un panel exclusivo para administradores donde concentré la edición profunda (CRUD) del sistema.

- Tiene un buscador multicriterio (por estado, área, ordenamiento por fecha y coincidencia de texto) que concatena dinámicamente condicionales `WHERE` y `execute()` paramétricos en PHP.
- Permite cambiar los archivos PDF ya subidos, alterar fechas, re-turnar a otra área y forzar estados técnicos a través de modales.

### 3.6. Generador de Hashes (`generar_hash.php`)

Dado que uso `password_hash()` en el login para máxima seguridad, desarrollé este pequeño script interno. Me permite (o al administrador de sistemas) ingresar una contraseña en texto plano y generar el hash encriptado (`$2y$10$...`) listo para copiar y pegar directamente en la base de datos o en la creación de usuarios.

## 4. Experiencia de Usuario (UX) e Interfaz (UI)

Me preocupé mucho por evitar errores humanos (de "dedo"). Por ello, implementé:

- **Modales de Error Proactivos:** En lugar de dejar que el backend rebote formularios incompletos, programé JavaScript (ej. `validarCaptura()`) que intercepta el envío (`e.preventDefault()`). Si faltan campos clave (número, asunto, área), despliego un Modal rojo estricto que obliga al usuario a llenar los datos.



**CIUDAD DE MÉXICO**  
CAPITAL DE LA TRANSFORMACIÓN

- **Interacciones Fluidas:** Uso **Collapse** de Bootstrap para esconder formularios complejos (como el buscador y duplicador de oficios) para no saturar la pantalla hasta que el usuario lo necesite.
- **Retroalimentación visual:** Todo cambio de estado (Concluido, Revisión) o prioridad cambia de color automáticamente gracias a lógica en PHP incrustada en las clases CSS, lo que hace que los usuarios puedan escanear visualmente la urgencia de su bandeja en segundos.

**Conclusión** La lógica detrás de todo este código fue construir un ecosistema seguro, donde la información es inmutable en cuanto a su trazabilidad, evitando duplicidad de folios y asegurando que las responsabilidades entre áreas queden perfectamente documentadas y medibles.